Proyecto · Parte 2
Implementación y despliegue seguro

Construir en la nube lo que diseñaron en papel

Mismo equipo de la Parte 1 · Entrega: **jueves 19 de noviembre** · Presentación en clase · **25 % de la nota**

Atributo de acreditación evaluado: **Herramientas de Ingeniería (HI)**

En una frase. Verdeazul aprobó el diseño que entregaron en octubre. Ahora hay que **construirlo y ponerlo en internet**, con los controles de seguridad que ustedes mismos prometieron.

1 Objetivo

Implementar el sistema diseñado en la Parte 1 y desplegarlo en infraestructura real, aplicando los controles de acceso y las políticas que documentaron. Al terminar, cualquier persona debe poder abrir el sistema desde internet por una dirección pública y segura.

Lo que se evalúa. No la cantidad de funcionalidad, sino que **lo construido corresponda al diseño** y que los controles de seguridad estén realmente configurados —no solo mencionados—.

2 Qué deben construir

2.1 Alcance funcional

1. **La matriz de control de acceso completa.** Todos los roles y todos los permisos que documentaron en la Parte 1 deben estar implementados y ser verificables. No es un subconjunto: es la matriz entera.
2. **Autenticación de usuarios**, con las credenciales almacenadas de forma segura.
3. **Persistencia en base de datos.** Los datos se guardan y se leen de PostgreSQL.
4. **Registro de auditoría.** Quién hizo qué y cuándo, almacenado.
5. **Emisión de certificados firmados digitalmente** (ver abajo).
6. **La arquitectura diseñada.** Los componentes que definieron en la Parte 1 deben existir como tales, con la misma separación de responsabilidades y las mismas fronteras de confianza entre ellos.

La interfaz queda a criterio de cada equipo. No se evalúa el diseño visual. Una interfaz sobria que permita ejercitar todos los permisos de la matriz es suficiente; inviertan el tiempo en la seguridad, no en la estética.

2.2 Certificados de carbono firmados

Cada certificado que emita el sistema debe ir **firmado digitalmente**, de modo que cualquiera pueda comprobar que no fue alterado y que lo emitió Verdeazul.

1. El sistema firma el certificado con una **clave privada** que vive protegida como cualquier otro secreto.
2. El certificado **lleva incrustada la clave pública** correspondiente.

3. Debe existir una forma de **verificar** un certificado: dado el archivo, confirmar si la firma es válida y si el contenido fue modificado.

Por qué así. Un certificado de carbono vale dinero real y circula fuera del sistema. La firma digital es lo que permite que un comprador en otro país confíe en el documento sin tener acceso a la base de datos de Verdeazul. Es el tema de las Clases 17–18 y 20, aplicado.

Sobre el alcance. Un sistema con la matriz completa y bien asegurado vale más que uno con más funciones y los permisos a medias. Si deben recortar, recorten en funcionalidad accesoria, nunca en la matriz ni en los controles.

2.3 Controles de seguridad obligatorios

1. **Contraseñas con hash lento.** `bcrypt`, `scrypt` o `Argon2`, con *salt*. Nunca en texto claro ni con un hash rápido.
2. **Control de acceso del lado del servidor.** Ocultar un botón en la interfaz no es un control: el servidor debe rechazar la operación.
3. **Secretos fuera del código.** Ninguna credencial, clave ni cadena de conexión dentro del repositorio.
4. **TLS en todo el tráfico.** El sistema solo responde por HTTPS.
5. **Validación de entradas.** Consultas parametrizadas; nada de concatenar lo que escribe el usuario.
6. **Firma digital de los certificados**, con la clave privada protegida y verificable con la clave pública que viaja en el documento.

3 Infraestructura: dónde y cómo se despliega

3.1 Las tres piezas

Componente	Qué hay que hacer
Neon PostgreSQL	Crear el proyecto y la base de datos. La cadena de conexión se guarda como secreto, **nunca** en el repositorio. La conexión viaja cifrada.
Google Cloud Run ejecución	Desplegar cada componente como **imagen de Docker**. Cloud Run asigna un dominio público (`*.run.app`) con **TLS automático**: esa es la dirección por la que se accede al sistema.
Secret Manager secretos	Las credenciales se guardan ahí y se inyectan al contenedor como variables de entorno. **No** se escriben en el `Dockerfile` ni en el código.
Artifact Registry imágenes	Las imágenes de Docker se publican en el registro de Google, **versionadas con etiquetas explícitas**. Cloud Run despliega desde ahí.

Por qué estas tecnologías. Las tres tienen capa gratuita suficiente para este proyecto, y reproducen cómo se despliega software real hoy: contenedores, base gestionada y secretos administrados aparte.

Solo se evalúan proyectos corriendo en la nube. Un sistema que funcione en la máquina de alguien, o en un servidor local, **no se evalúa**. El criterio es simple: si el profesor no puede abrirlo desde internet, no existe.

Cuidado con el consumo. El uso de Cloud Run y Neon **debe mantenerse dentro de la capa gratuita**. No saturen la aplicación con pruebas de carga, procesos en bucle ni datos masivos: podrían generar un cobro inesperado. Si necesitan probar volumen, consulten antes.

3.2 Docker y el registro de imágenes

Cada componente de su arquitectura se empaqueta en su propia imagen. El `Dockerfile` debe estar versionado en el repositorio y la imagen debe construir sin intervención manual.

Las imágenes se publican en **Artifact Registry** de Google, y desde ahí las despliega Cloud Run. El versionado debe ser **explícito y rastreable**:

- Cada imagen lleva una **etiqueta de versión** propia (`v1.0.0`, `v1.1.0`...) o el *hash* del commit.
- **No basta con** `latest`: esa etiqueta se mueve y no permite saber qué está corriendo en producción.
- Debe poder identificarse **qué versión de la imagen** corresponde a qué estado del código.

Por qué importa el versionado. Si mañana aparece una vulnerabilidad, hay que poder responder dos preguntas: *¿qué versión está corriendo?* y *¿a cuál volvemos?* Con `latest` ninguna de las dos tiene respuesta.

Un detalle que se revisa. El contenedor **no debe ejecutarse como root**. Es una línea en el `Dockerfile` y es un control de mínimo privilegio aplicado a infraestructura.

3.3 Dependabot

Activen **Dependabot** en el repositorio de GitHub (`Settings → Code security`). Debe quedar configurado para revisar dependencias y, si aparecen alertas durante el proyecto, **atiendan al menos una** y documéntenlo.

Por qué importa. Las dependencias desactualizadas son una de las vías de ataque más comunes, y nadie las revisa a mano. Automatizar esa vigilancia es parte de la higiene de seguridad de cualquier proyecto serio.

4 Relación con la Parte 1

El sistema debe implementar **el diseño que ustedes entregaron**, no otro. En particular:

- Los **roles y permisos** deben corresponder a su matriz de control de acceso.
- El **modelo de control de acceso** que eligieron (RBAC, ABAC o híbrido) debe ser el que se implemente.
- Las **políticas** que documentaron deben tener reflejo técnico: si escribieron «se revocan los accesos el mismo día», debe existir la forma de revocarlos.
- La **arquitectura** debe reconocerse en el despliegue: si separaron la aplicación en componentes, esos componentes deben existir; si definieron una frontera de confianza entre dos de ellos, debe estar implementada y no ser solo un dibujo.

4.1 Si cambiaron de opinión

Es normal que al construir aparezcan problemas que el diseño no previó. **Pueden desviarse del diseño original**, pero cada desviación debe quedar registrada en el informe: qué cambiaron, por qué, y qué implicación de seguridad tuvo.

Esto suma, no resta. Documentar una desviación bien argumentada demuestra criterio de ingeniería. Lo que resta es que el sistema no se parezca al diseño y nadie lo explique.

5 Entregables

#	Entregable	Qué debe contener
1	**Sistema desplegado**	URL pública en Cloud Run, funcionando, con credenciales de prueba para cada rol.
2	**Repositorio en GitHub**	Código, `Dockerfile` de cada componente, Dependabot activo y `README` con instrucciones de despliegue y las etiquetas de versión publicadas.
3	**Informe de implementación**	6 a 10 páginas. Contenido en la sección siguiente.
4	**Presentación en clase**	19 de noviembre. 15 minutos: recorrido del sistema y de los controles. Todo el equipo participa.

5.1 Contenido del informe

1. **Arquitectura desplegada.** Diagrama de lo que realmente construyeron: componentes, imágenes y cómo se comunican.
2. **Controles implementados.** Por cada control obligatorio: dónde está en el código y cómo se verifica que funciona.
3. **Gestión de secretos.** Qué secretos existen, dónde viven y cómo llegan al contenedor.
4. **Seguridad heredada de la plataforma.** Qué protege Neon por defecto —sin que ustedes hagan nada— y qué quedó bajo su responsabilidad. Documenten también **qué opciones de seguridad eligieron** al configurar el proyecto y por qué (ver abajo).
5. **Desviaciones respecto al diseño.** Qué cambió y por qué.
6. **Cobertura de la matriz.** Tabla que muestre, permiso por permiso, dónde está implementado y cómo se comprobó.
7. **Evidencia.** Capturas de: el sistema funcionando por HTTPS, la configuración de Secret Manager, Dependabot activo, un certificado verificado correctamente y el intento fallido de un rol al hacer algo que no le corresponde.

5.2 Sobre la seguridad heredada de la plataforma

Usar un servicio gestionado significa que **parte de la seguridad ya viene resuelta** —y parte sigue siendo suya—. Saber dónde está esa frontera es una competencia de ingeniería, no un trámite.

Para **Neon**, documenten al menos:

- **Qué protege la plataforma por defecto:** cifrado en reposo, cifrado en tránsito, aislamiento del proyecto, respaldos y recuperación a un punto en el tiempo, autenticación de la conexión.

- **Qué configuraron ustedes:** el modo de conexión exigido, si restringieron direcciones de origen, qué usuario de base de datos usa la aplicación y con qué permisos, si crearon ramas o entornos separados.
- **Por qué eligieron esa configuración** y qué alternativa descartaron.
- **Qué queda sin cubrir.** Ningún proveedor protege contra una consulta mal construida, un permiso excesivo o una credencial filtrada. Nombren explícitamente qué riesgos siguen siendo suyos.

El modelo de responsabilidad compartida. Es el mismo razonamiento para Cloud Run: Google protege la infraestructura y termina el TLS; ustedes responden por el código, los permisos y los secretos. Media página basta —lo que se evalúa es que sepan trazar la línea, no la extensión—.

Un error frecuente. Escribir «Neon es seguro» como si eso cerrara el tema. La pregunta correcta es *seguro contra qué*, y qué amenaza del modelo de la Parte 1 **no** queda cubierta por la plataforma.

6 Presentación en clase

El **19 de noviembre** cada equipo presenta en clase, con el sistema corriendo en su URL pública. Quince minutos. Deben mostrar:

1. El sistema respondiendo por **HTTPS** desde internet.
2. Un usuario de cada rol **haciendo lo que le corresponde**.
3. Un usuario **intentando lo que no le corresponde**, y el sistema rechazándolo.
4. La **emisión de un certificado firmado** y su **verificación** —incluido el caso de un certificado alterado, que debe fallar—.
5. El **registro de auditoría** con esas acciones.
6. La configuración de **secretos** y de **Dependabot**.

Pregunta que va a salir. A cada integrante se le puede pedir que explique una parte del sistema que no escribió. Repartirse el trabajo está bien; desconocer lo que hizo el resto, no.

7 Evaluación

Criterio	Puntos	Qué se busca
Despliegue funcional en la nube El sistema abre desde internet por HTTPS, en Cloud Run, con base en Neon.	16	Que funcione de verdad, no en la máquina de alguien.

Criterio	Puntos	Qué se busca
Matriz de acceso y arquitectura implementadas Todos los roles y permisos de la Parte 1 implementados; el servidor rechaza lo no autorizado. Los componentes, su separación y la forma en que se comunican corresponden a la arquitectura diseñada.	25	Que el control viva en el servidor, no en la interfaz, y que el sistema desplegado tenga la estructura que diseñaron.
Certificados firmados Firma con clave privada protegida; clave pública en el certificado; verificación funcional.	15	Que un tercero pueda confiar sin acceso al sistema.
Gestión de secretos Secret Manager configurado; nada sensible en el repositorio.	12	Que no haya credenciales versionadas.
Contenedores, registro y Dependabot Imágenes en Artifact Registry con versiones explícitas; contenedor sin root; Dependabot activo.	10	Higiene de infraestructura y trazabilidad.
Correspondencia con la Parte 1 Lo construido refleja el diseño; las desviaciones están justificadas.	7	Que el diseño haya servido para algo.
Informe y evidencia Tabla de cobertura de la matriz, análisis de la seguridad heredada de la plataforma y capturas que prueban cada control.	10	Que se pueda verificar sin pedirles nada, y que sepan trazar la frontera de responsabilidad.
Presentación y defensa individual Todo el equipo conoce el sistema completo.	5	Que nadie llegue a mirar.
Total	**100**	

7.1 Condiciones que anulan puntos

- **Credenciales en el repositorio:** se pierde el total del criterio de secretos, aunque el sistema funcione.
- **Contraseñas sin hash o con hash rápido:** se pierde la mitad del criterio de control de acceso.
- **El sistema no abre desde internet el día de la presentación:** se pierde el criterio de despliegue. Prueben desde una red distinta a la suya antes de presentar.
- **Imágenes publicadas solo como `latest`:** se pierde la mitad del criterio de contenedores.
- **Permisos de la matriz sin implementar:** se descuenta proporcionalmente del criterio de control de acceso.

8 Recursos

- **Neon** —PostgreSQL gestionado— `https://neon.tech`. Capa gratuita suficiente para este proyecto.
- **Google Cloud Run** —`https://cloud.google.com/run`. Incluye crédito inicial y capa gratuita mensual.

- **Secret Manager** —https://cloud.google.com/secret-manager.
- **Artifact Registry** —https://cloud.google.com/artifact-registry.
- **Dependabot** —https://docs.github.com/code-security/dependabot.
- **OWASP Cheat Sheet Series** —referencia de controles: https://cheatsheetseries.owasp.org.

Sobre los costos. Todo lo necesario cabe en las capas gratuitas. Si algún servicio les pide tarjeta, avisen **antes** de registrarse: hay alternativas y cuentas educativas.

9 Trabajo en grupo

Mismo equipo de la Parte 1. El trabajo se reparte como ustedes decidan, pero la defensa es individual: cada quien responde por el sistema completo.

Un consejo de orden. Despliegen algo mínimo la primera semana —aunque solo muestre «hola»— y después agréguenle funcionalidad. Dejar el despliegue para el final es la forma más común de no llegar.